AWS Level 3 iteration 1 terminal documentation

Claudio Verdi | APL | 23 October 2026

I created an Ubuntu server in AWS using Terraform, installed Nginx over SSH and deployed a simple website. I verified the page on the server, from my computer and in a browser. After testing, I removed the instance with Terraform and verified its terminated state using AWS CLI.

Environment and resources

Property	Value
Region	eu-north-1
IAM	tabish-admin
Terraform / AWS provider	1.16.3 / hashicorp/aws 6.70.0
Instance ID	i-014006d7d2192e50c
Instance type	t3.micro
AMI	ami-06fb1802e2b757f09
OS	Ubuntu 24.04.5 LTS x86_64
VPC	vpc-00dcc7fb53fe9926b
Subnet	subnet-05d4dca4820d2eb94 / eu-north-1a
Security group	sg-0fb87d0f08a213466
Key pair	devops.school.level3.tabish-admin
Former public IP	16.170.140.221
Private IP	172.31.31.13
Nginx / Git	1.24.0 / 2.43.0

Reference: Level3 Iteration1 (1)(2)(1).sh. I followed the workflow step by step, adapting it to the actual environment.

Preparation and AWS identity

I checked the tools in Ubuntu under WSL. Terraform 1.16.3 and AWS CLI were installed. jq was missing, so I installed it with apt. STS confirmed access as the IAM user tabish-admin.

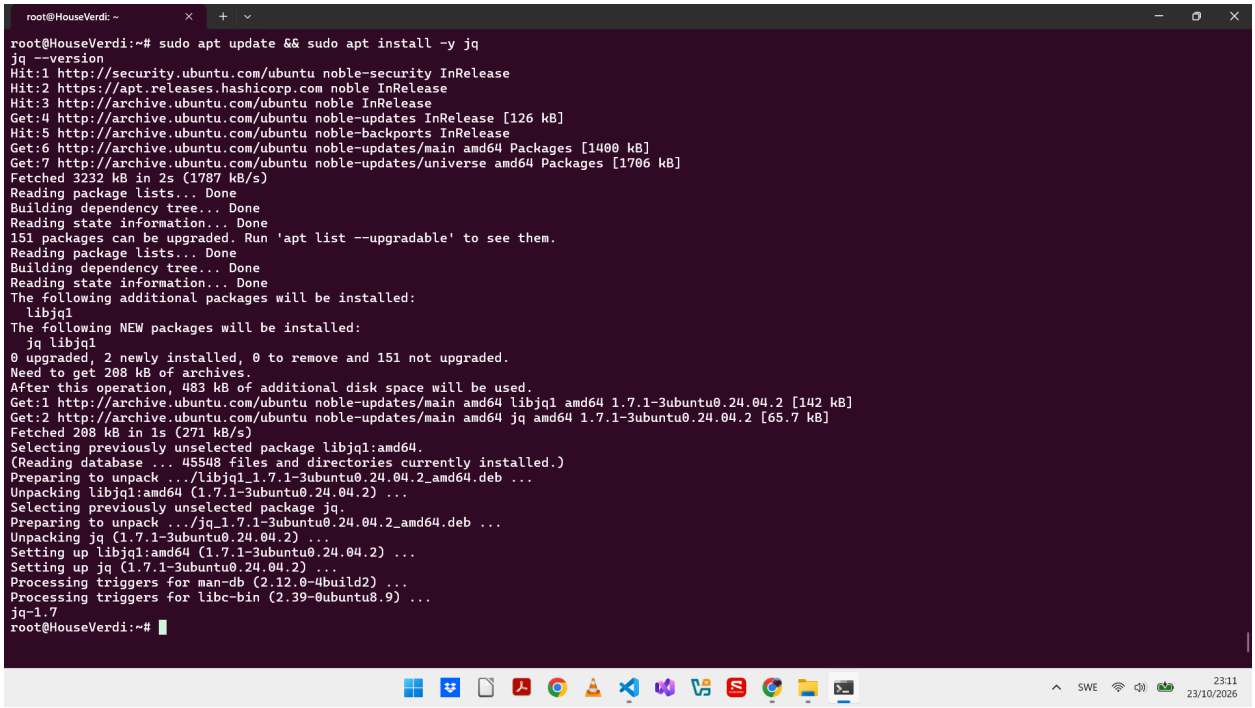


Figure 1. Completed jq installation

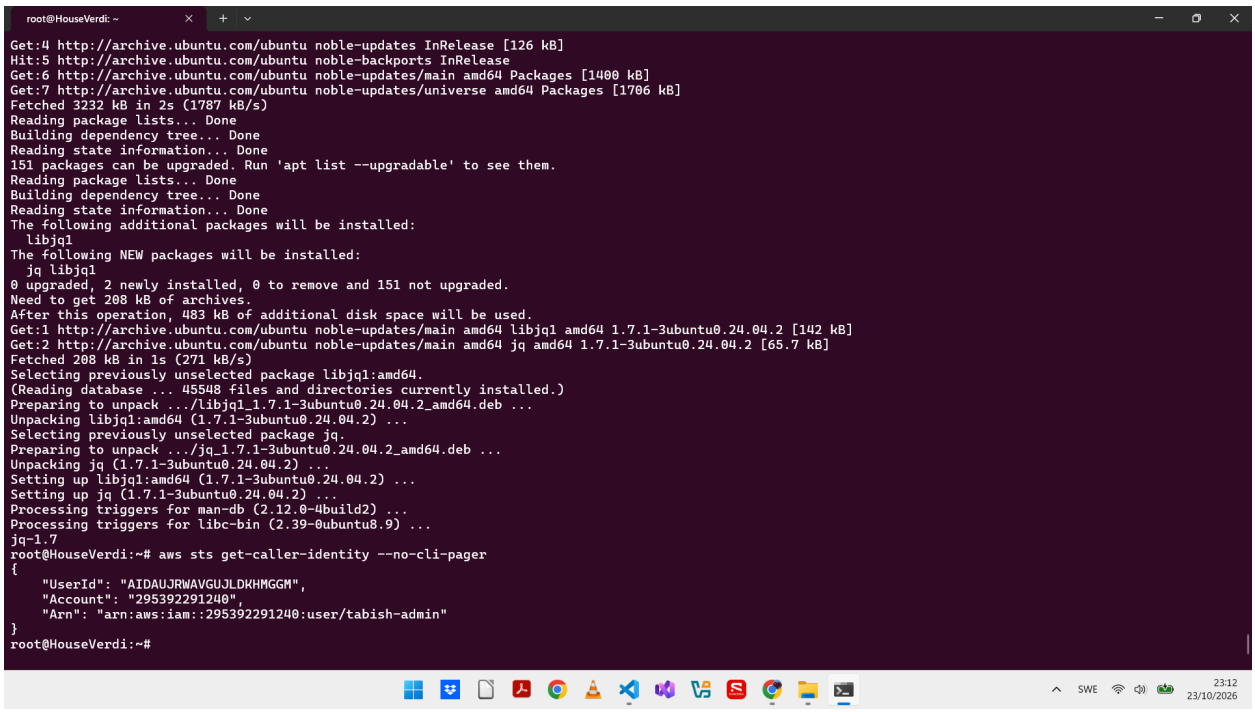


Figure 2. AWS identity verified with STS

SSH key and network

I reused the existing local key ~/.ssh/id_rsa_level3 and imported its public part into AWS. The private file had permission 600 and remained on my computer. I used the default VPC and selected the subnet in eu-north-1a.

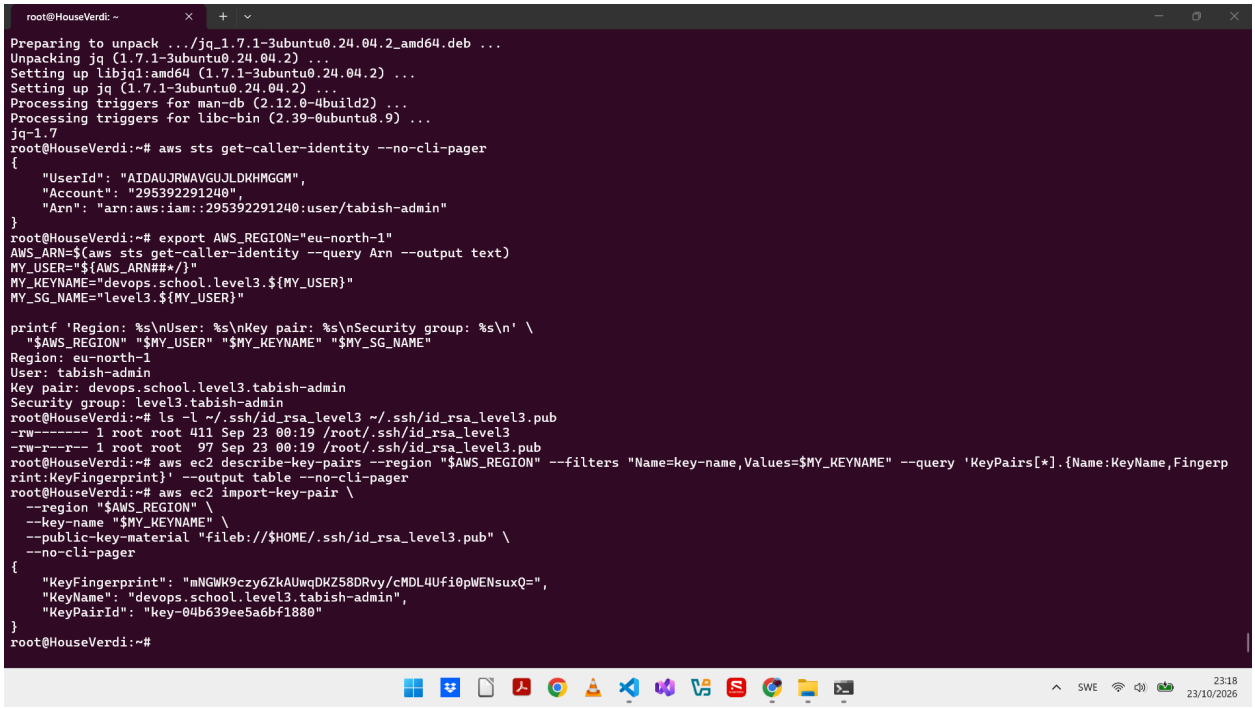


Figure 3. Public key imported into AWS

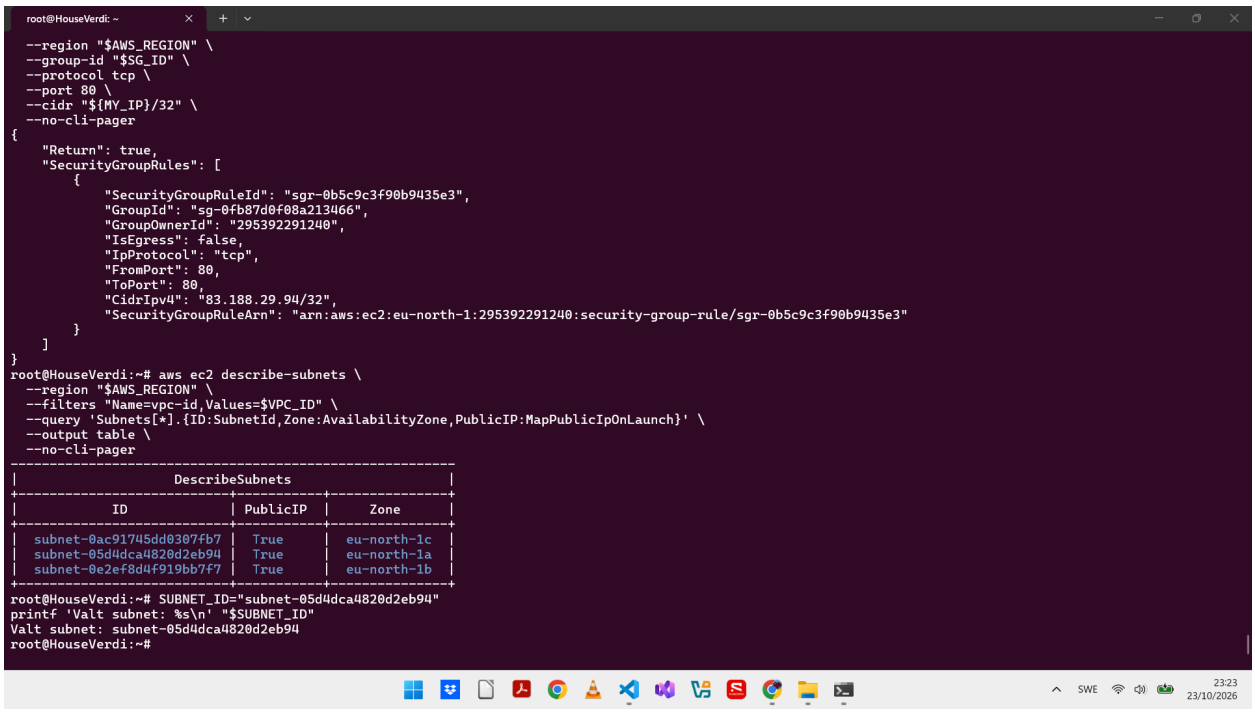


Figure 4. Selected subnet in eu-north-1a

Security group and Ubuntu image

I created the level3.tabish-admin security group and allowed TCP ports 22 and 80 from my public IP at the time, 83.188.29.94/32. Both SSH and HTTP were restricted to that address. I retrieved the Ubuntu image using AWS CLI and Canonical owner ID 099720109477.

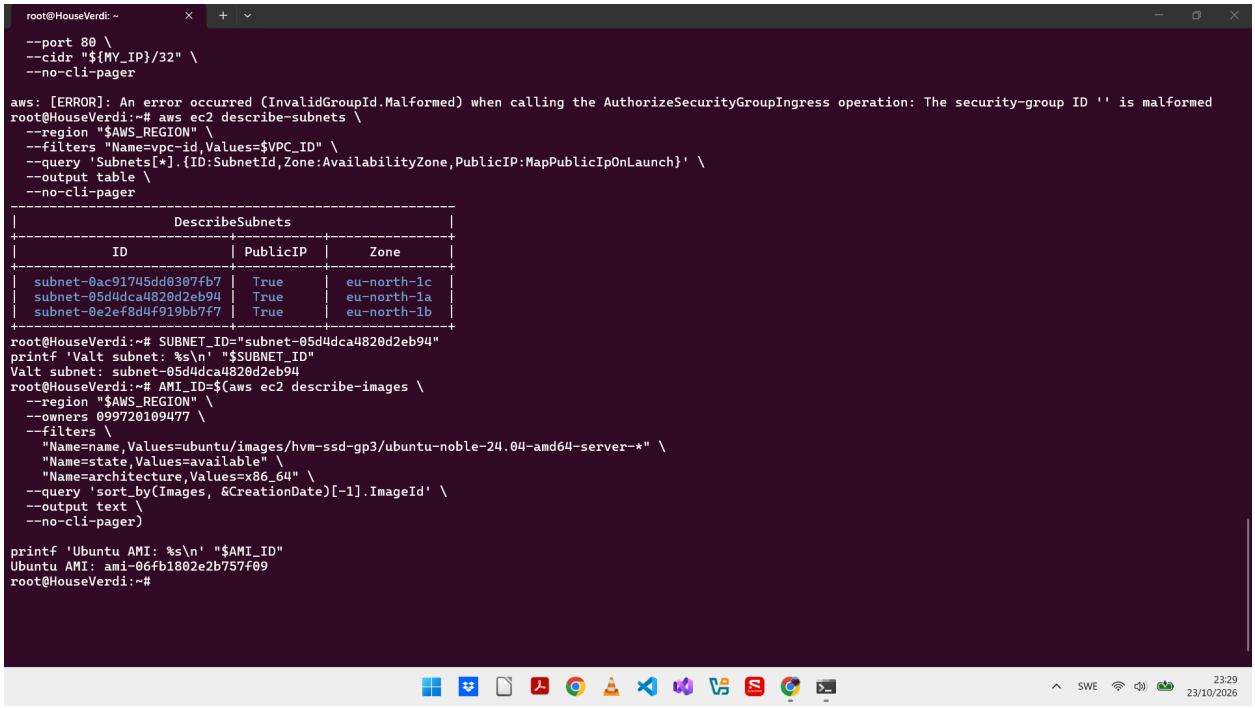


Figure 5. AMI result and failed repeated command with an empty SG_ID

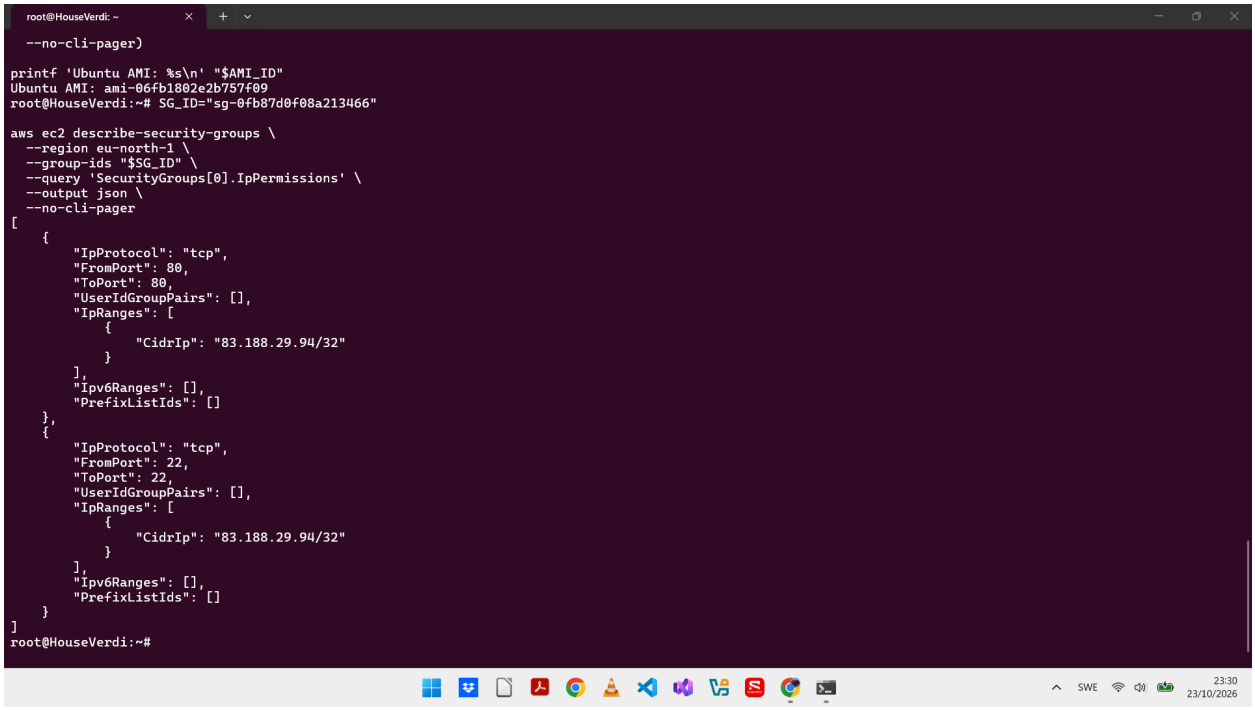


Figure 6. Restored SG_ID and verified port 22 and 80 rules

Terraform configuration and initialization

To preserve older files, I created ~/aws-devops-level3-iteration1. I split the configuration into providers.tf, variables.tf, main.tf, outputs.tf and terraform.tfvars. The complete files appear in the appendix.

Terraform managed one EC2 instance. The VPC and subnet already existed; I prepared the key pair and security group with AWS CLI. The instance used a public IP, required IMDSv2 and had an encrypted root disk.

```
terraform fmt
terraform init
terraform validate
```

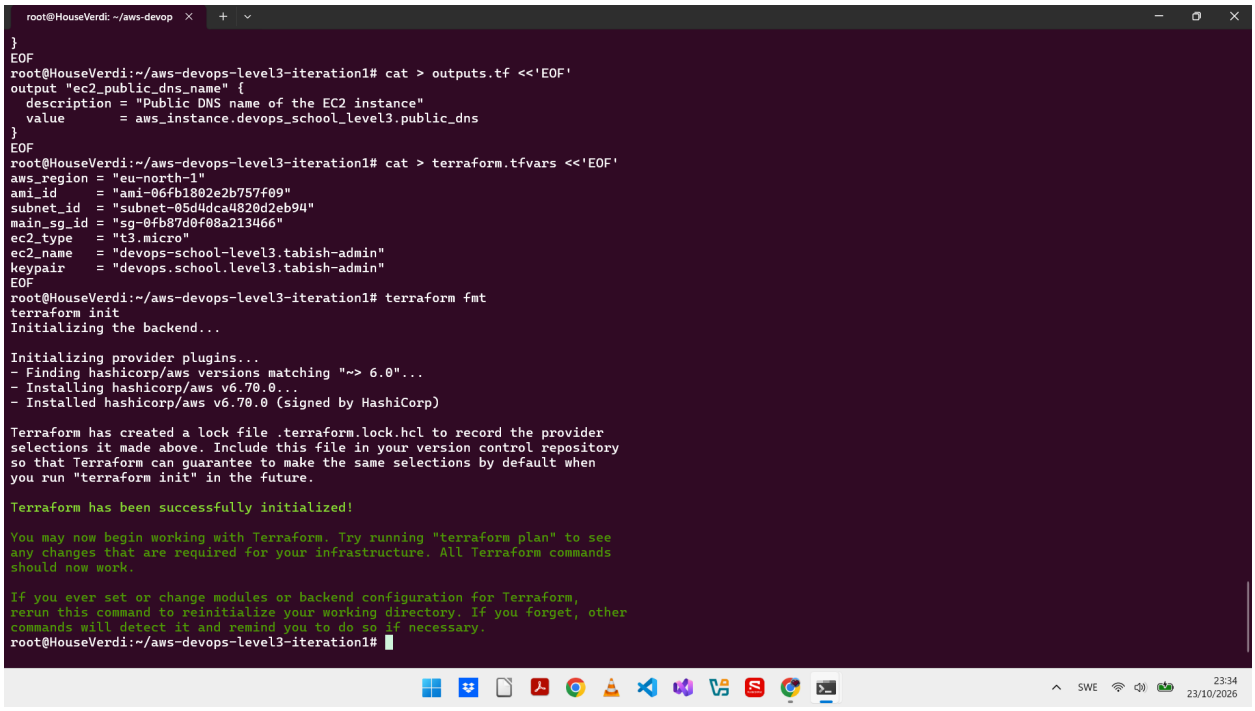


Figure 7. Initialization installed AWS provider 6.70.0 and created a lock file

terraform validate returned Success! The configuration is valid. I then saved the plan using terraform plan -out=tfplan.

Planning and creating the instance

The plan showed 1 to add, 0 to change and 0 to destroy. I applied the saved plan using terraform apply tfplan. The instance was created and Terraform returned its public DNS name.

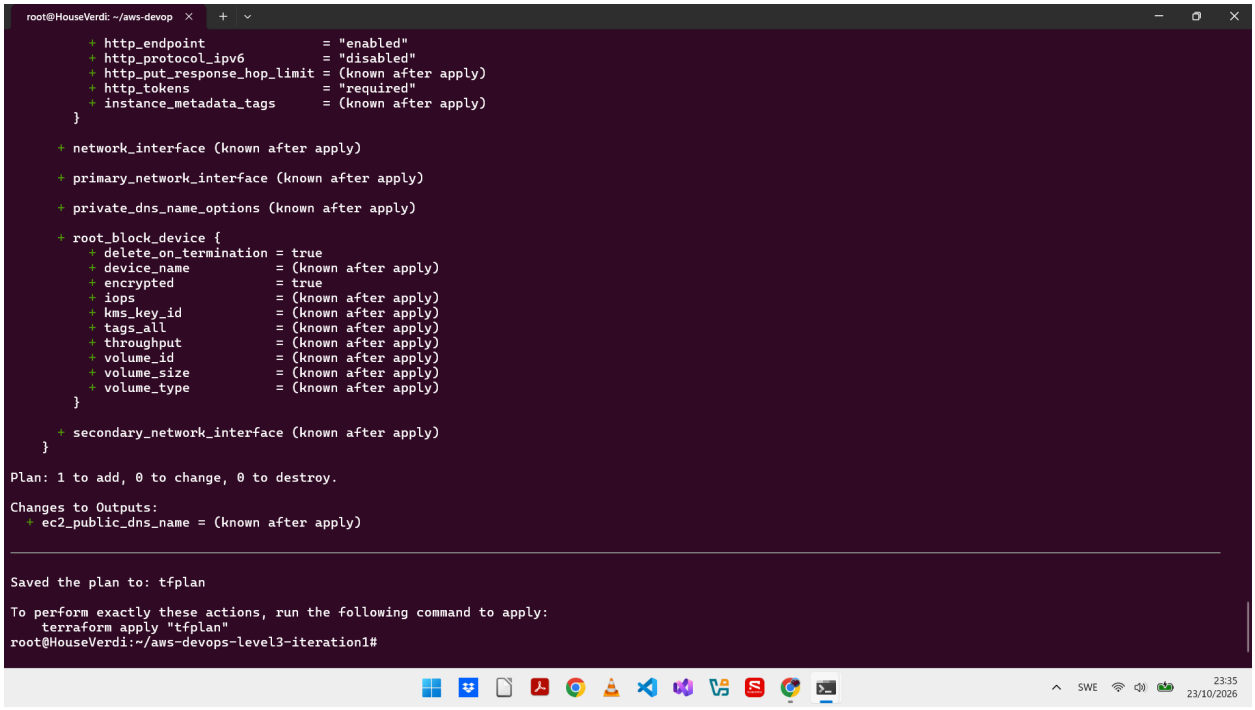


Figure 8. Saved plan with one new resource

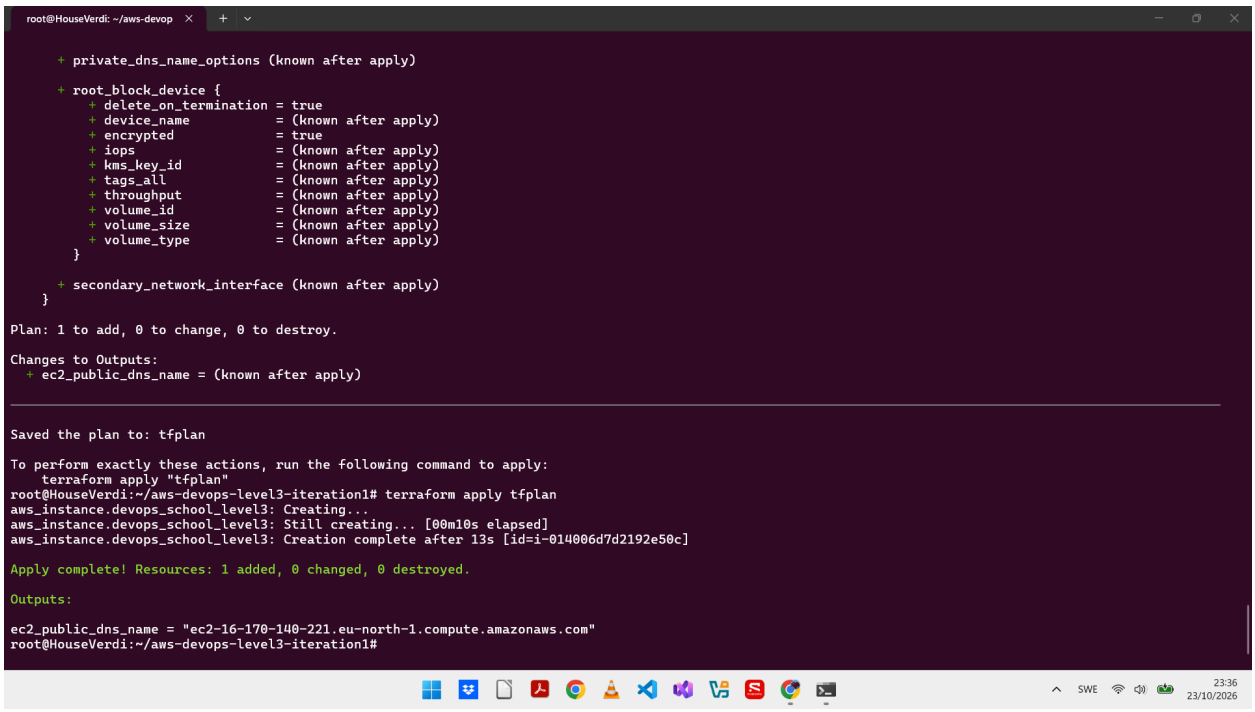


Figure 9. Successful apply with instance ID and public DNS name

SSH connection and system update

I retrieved the DNS name from Terraform and connected as ubuntu using my local private key. The server login banner showed Ubuntu 24.04.5 LTS. The update command completed with 0 packages upgraded.

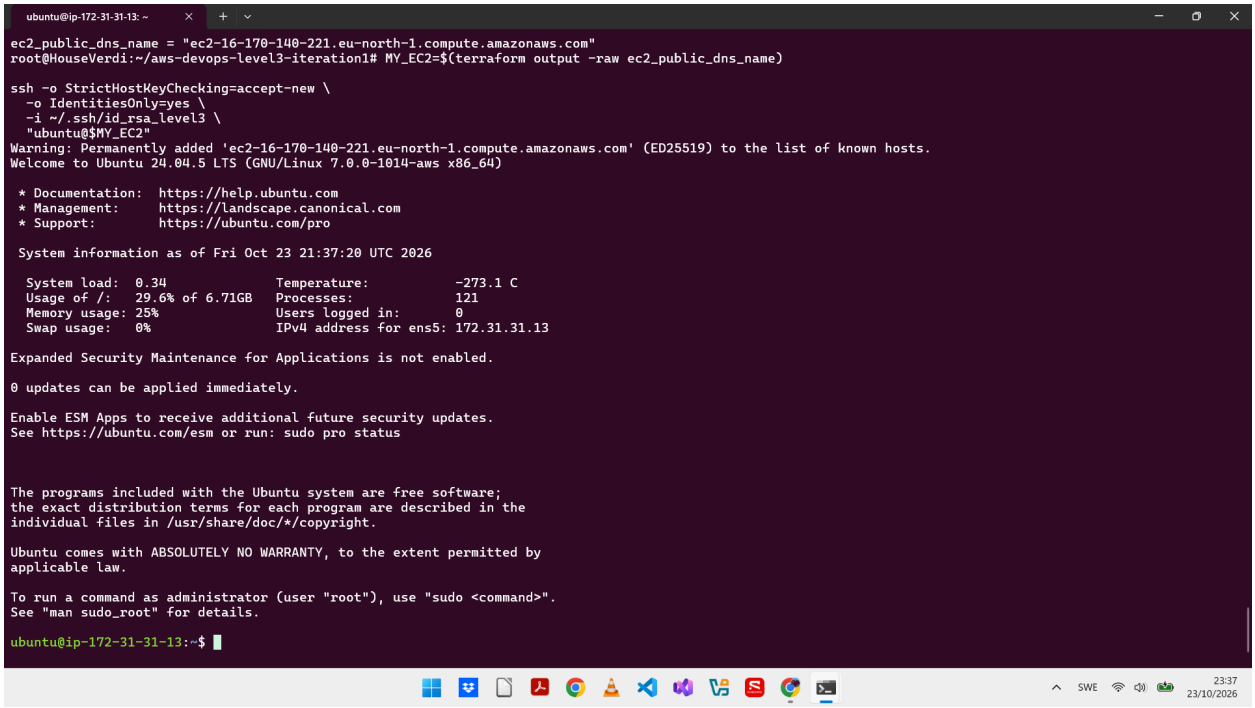


Figure 10. Successful SSH login to Ubuntu

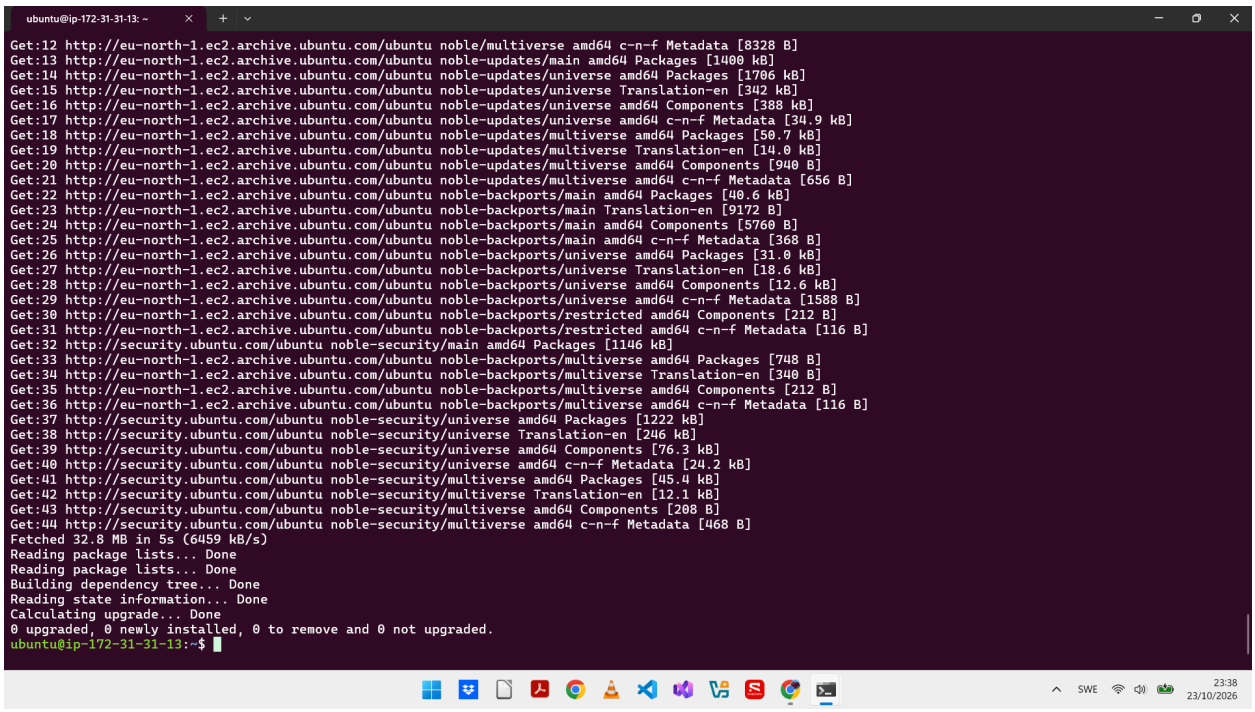


Figure 11. System package lists and upgrades checked

Installing Nginx and retrieving the website

I installed Nginx and Git using apt. I enabled Nginx with systemctl enable --now nginx; the checks returned active and enabled. I then cloned the website files and inspected the directory.

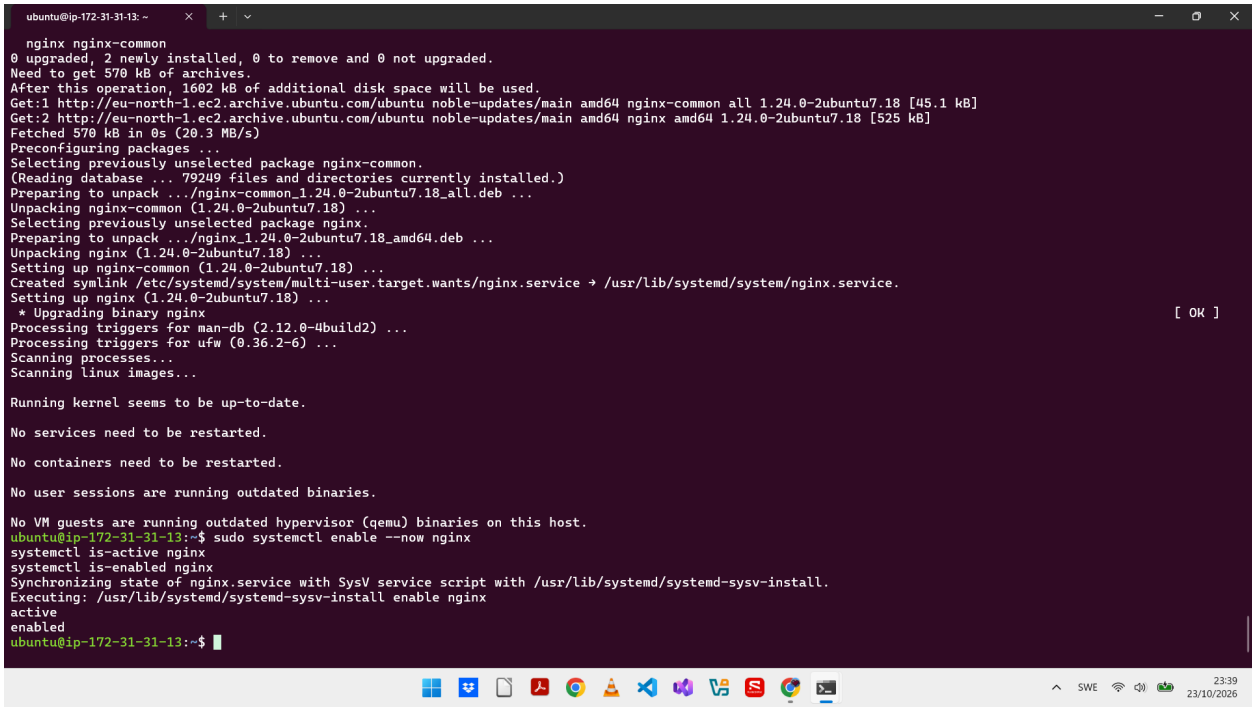


Figure 12. Nginx is active and enabled for automatic startup

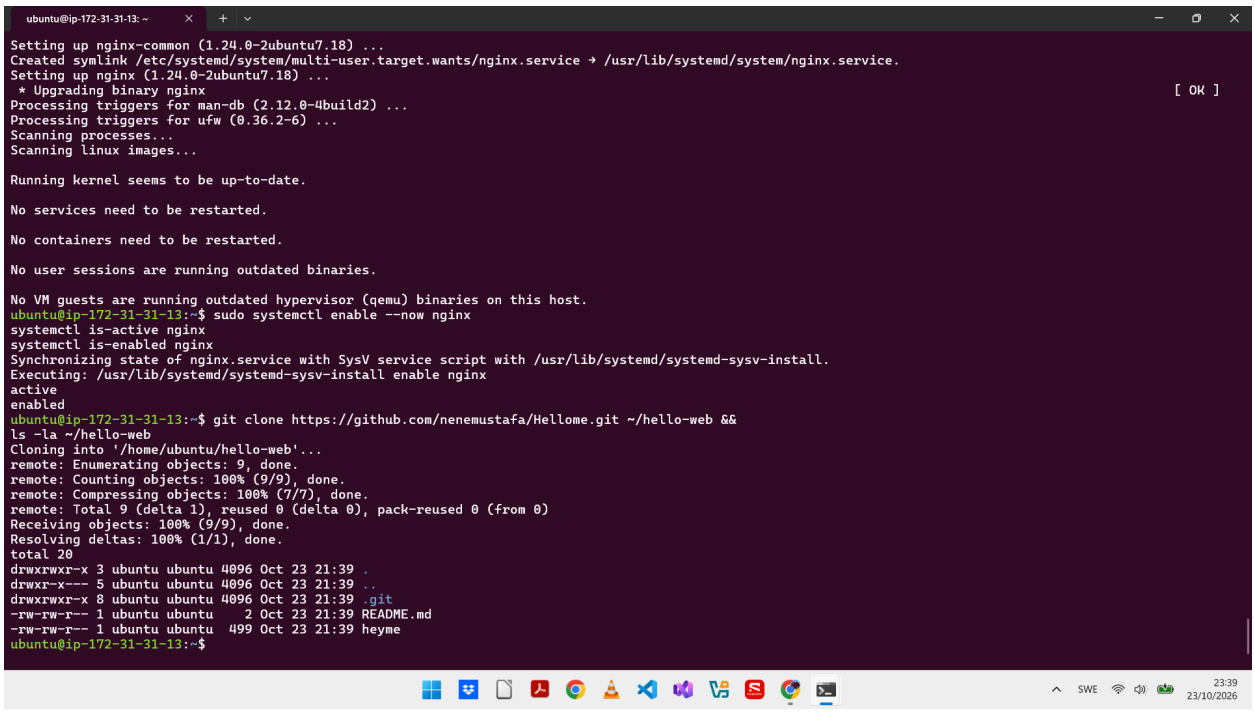


Figure 13. The repository contained the 499-byte file heyme

Deployment and local HTTP test

The html directory expected by the guide was absent. The website HTML was in heyme, which I copied to /var/www/html/index.html. nginx -t succeeded and I reloaded the service. curl against localhost returned HTTP 200 and the page HTML with the heading Hey me!.

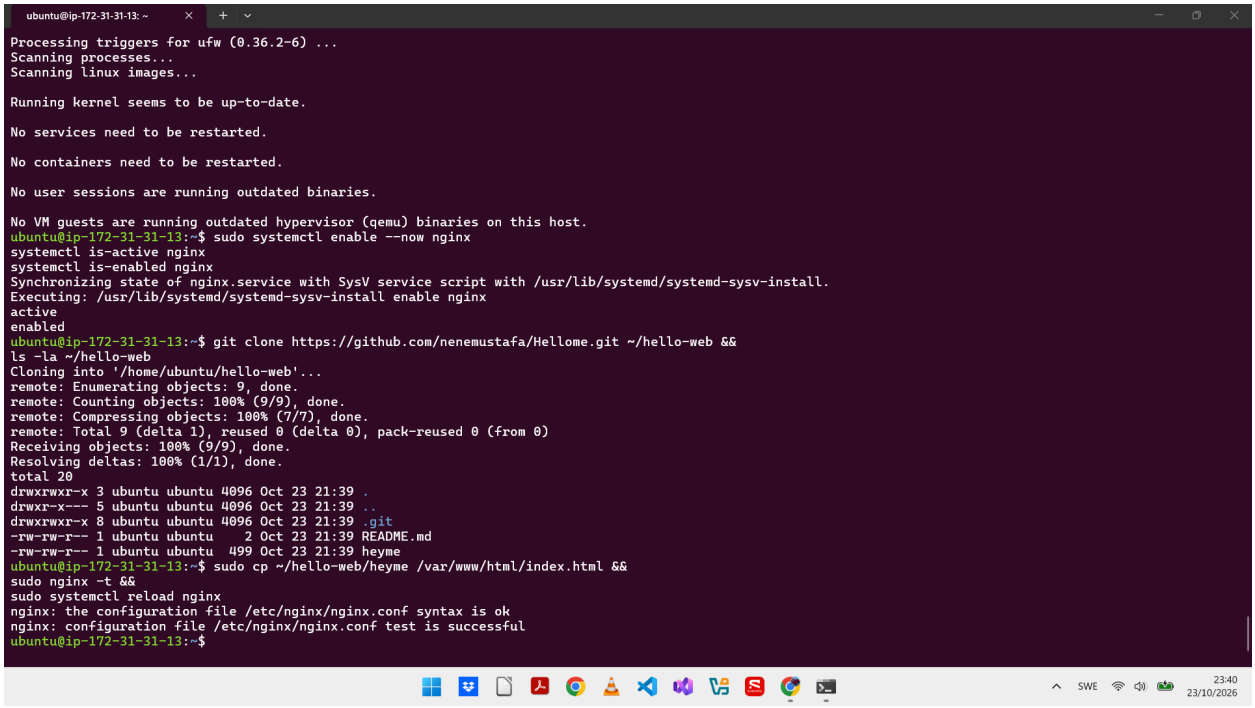


Figure 14. Copy to index.html and successful configuration test

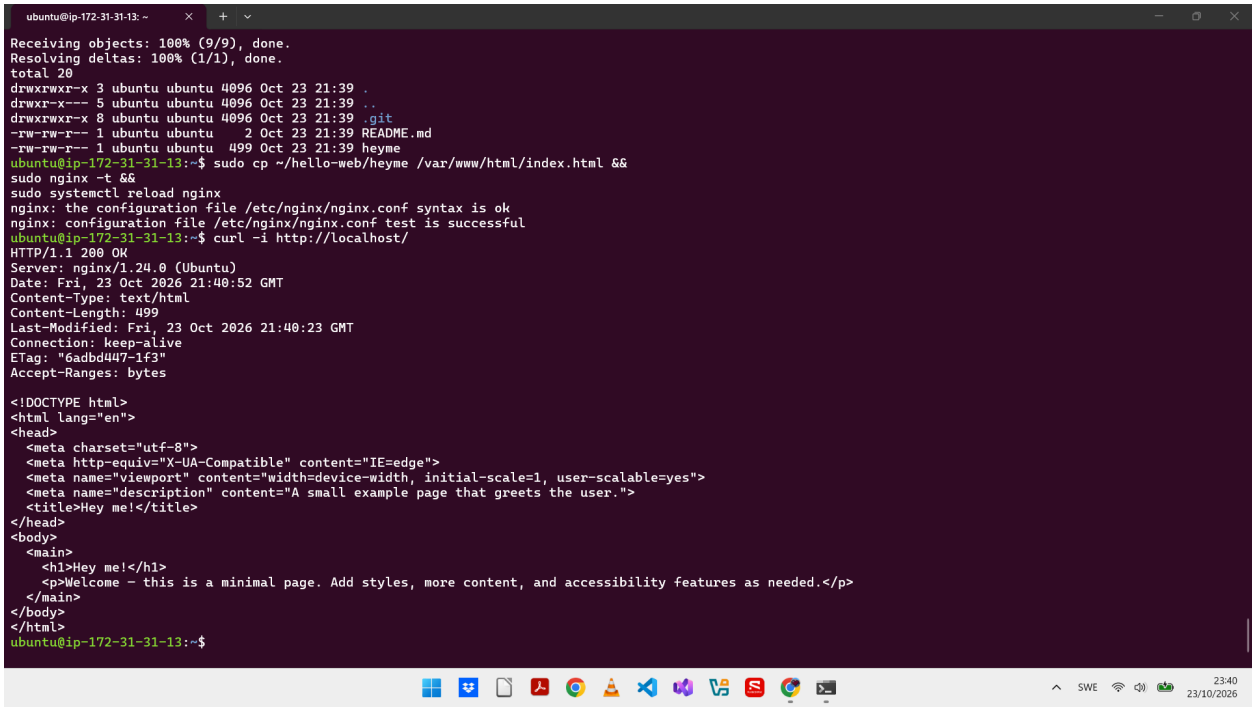


Figure 15. HTTP 200 from localhost with the expected HTML

Browser and external HTTP test

The browser displayed the page at http://16.170.140.221/. After leaving the SSH session, I ran curl from my local WSL terminal against the instance DNS name. That test also returned HTTP 200 and the Hey me! content.

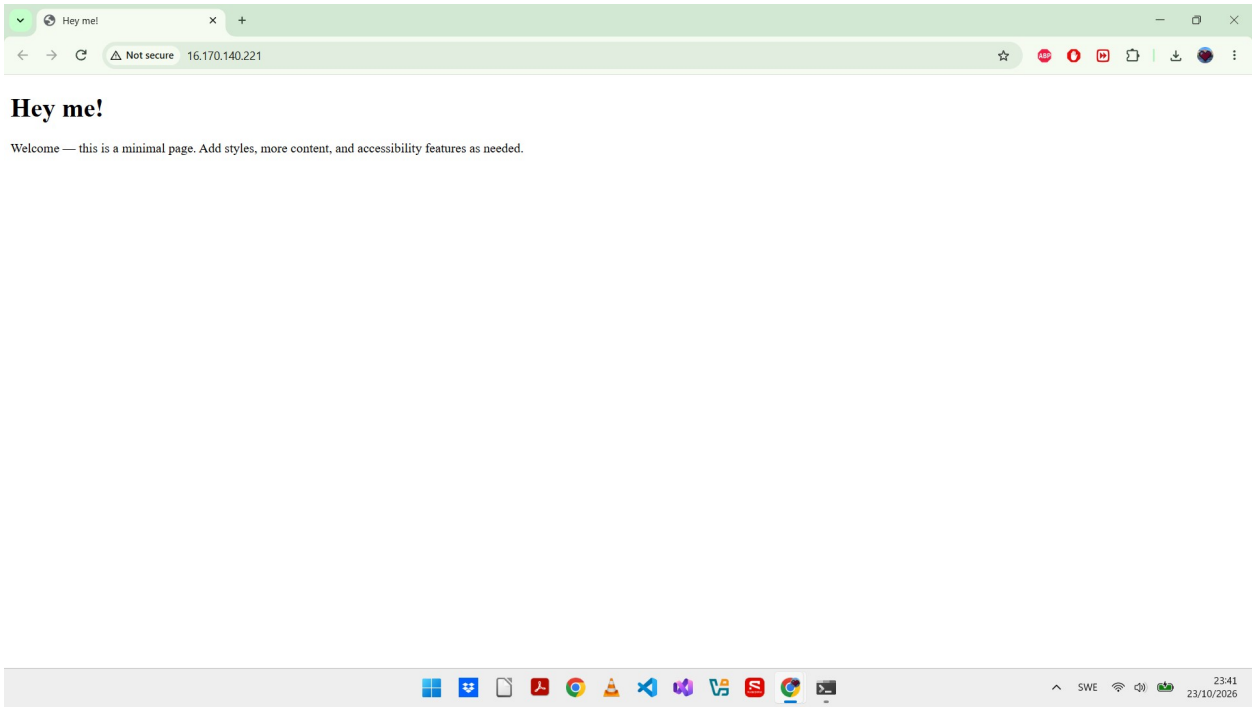


Figure 16. Website at the instance public IP

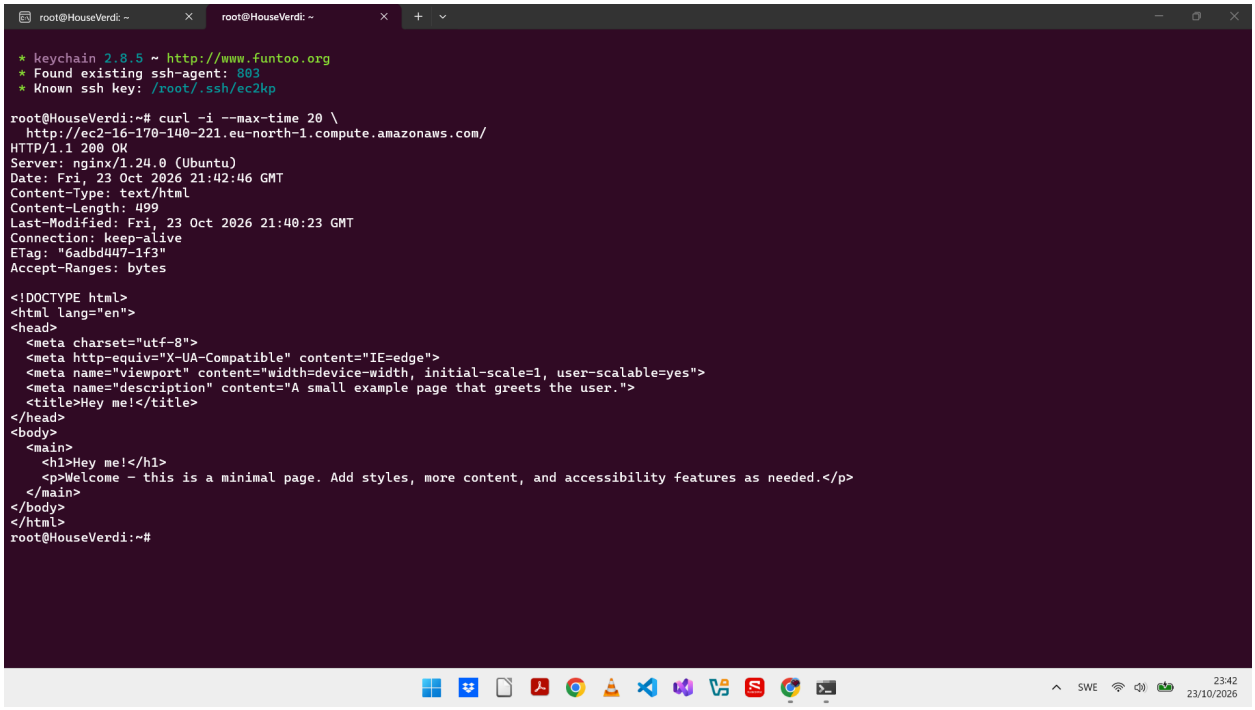


Figure 17. HTTP 200 and expected HTML from the local computer

Terraform verification and removal

After deployment, I ran terraform plan again. The No changes result showed that the infrastructure matched the configuration. I then saved a separate removal plan using terraform plan -destroy -out=destroyplan.

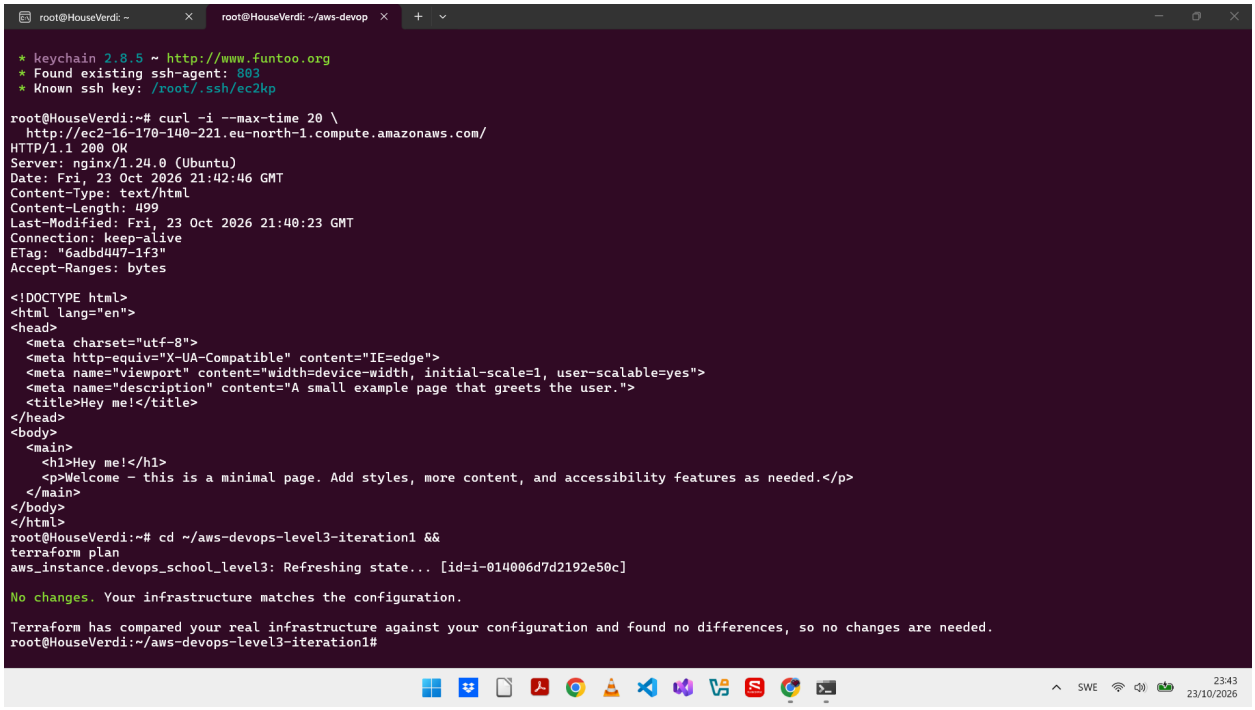


Figure 18. Terraform shows No changes after deployment

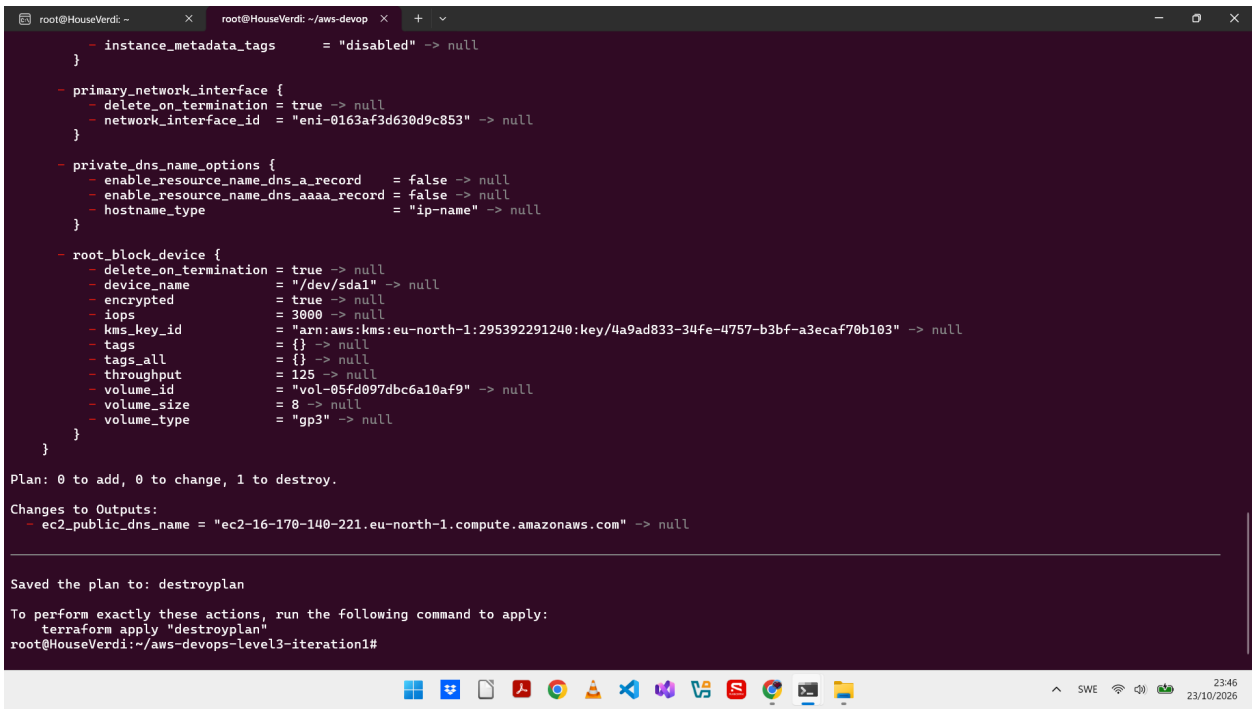


Figure 19. Removal plan with one resource and encrypted root disk

Verified cleanup

I ran terraform apply destroyplan. Terraform reported 0 added, 0 changed, 1 destroyed. A separate AWS CLI check of i-014006d7d2192e50c returned terminated on 23 October 2026 at 23:49 Swedish time.

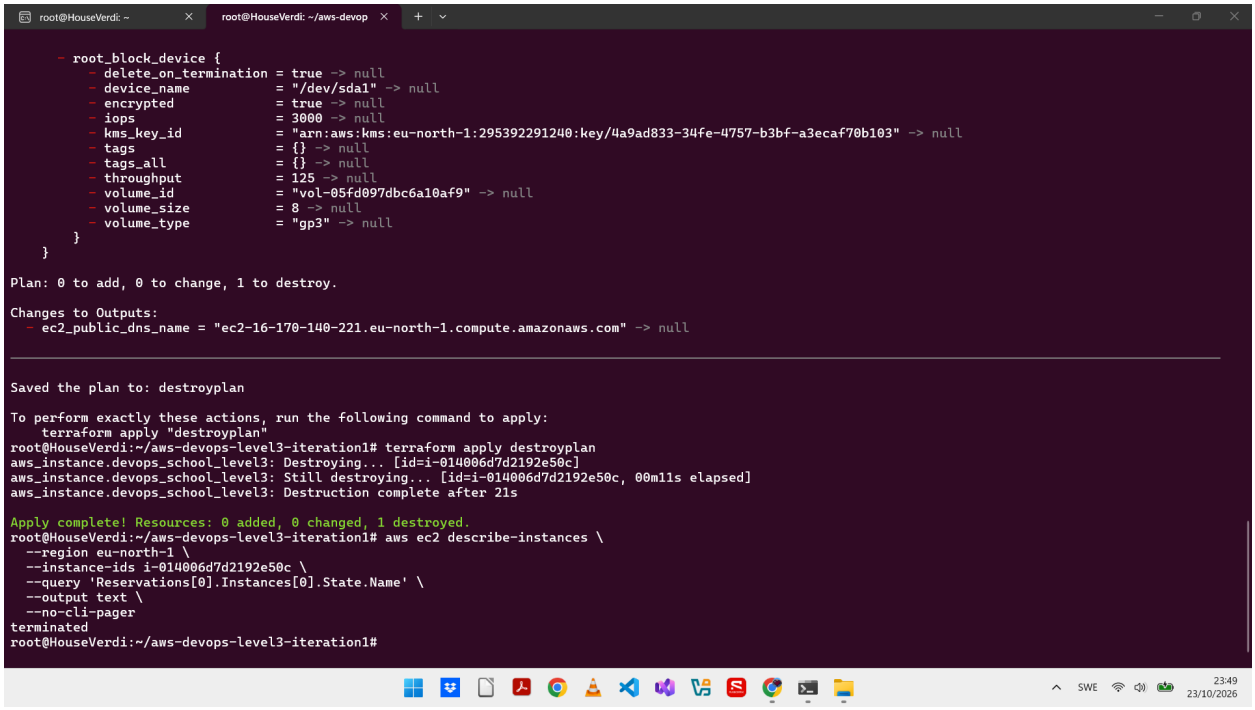


Figure 20. Completed removal and verified final state terminated

```
terraform apply destroyplan
aws ec2 describe-instances --region eu-north-1 \
  --instance-ids i-014006d7d2192e50c \
  --query 'Reservations[0].Instances[0].State.Name' \
  --output text --no-cli-pager
```

The root disk was configured with delete_on_termination=true. The key pair and security group were created outside Terraform and were not included in this removal. The former website IP and DNS document the tests before cleanup.

Problems improvements and conclusion

Problems and solutions

jq was missing locally and was installed. A repeated AWS command returned InvalidGroupId.Malformed because SG_ID was empty in that terminal session. I restored its value and verified that both inbound rules already existed.

The guide's file structure did not match the cloned repository. I inspected the files and deployed heyme as index.html. The actual page text was Hey me! with a lowercase m; the web tests document that text.

Improvements

Future improvements could include managing the security group and key pair in Terraform, automating installation and deployment, and adding HTTPS. Nginx was enabled, but I did not perform a reboot test in this iteration.

Conclusion and evidence

The work demonstrates Terraform provisioning, SSH access, manual deployment, successful HTTP tests and verified removal. The Terraform plan checks infrastructure; Nginx and the web content were verified separately.

Evidence consists of the reference script Level3 Iteration1 (1)(2)(1).sh, embedded screenshots from 23 October 2026, and level3-1-terraform.txt, level3-1-terminal-history.txt and level3-1-server-history.txt. The local terminal history also contains older commands; only the current sequence is used here.

Appendix Terraform files

The files below are reproduced from the exported configuration. terraform.tfvars must remain separate from the .tf files.

providers tf

```
terraform {
  required_version = ">= 1.5.0"

  required_providers {
    aws = {
      source  = "hashicorp/aws"
      version = "~> 6.0"
    }
  }
}

provider "aws" {
  region = var.aws_region
}
```

variables tf

```
variable "aws_region" {
  type = string
}

variable "ami_id" {
  type = string
}

variable "subnet_id" {
  type = string
}

variable "main_sg_id" {
  type = string
}

variable "ec2_type" {
  type = string
}

variable "ec2_name" {
  type = string
}

variable "keypair" {
  type = string
}
```

Appendix instance and variable values

main tf

```
resource "aws_instance" "devops_school_level3" {
  ami                         = var.ami_id
  instance_type               = var.ec2_type
  subnet_id                   = var.subnet_id
  key_name                    = var.keypair
  vpc_security_group_ids      = [var.main_sg_id]
  associate_public_ip_address = true

  metadata_options {
    http_tokens = "required"
  }

  root_block_device {
    encrypted = true
  }

  tags = {
    Name = var.ec2_name
  }
}
```

outputs tf

```
output "ec2_public_dns_name" {
  description = "Public DNS name of the EC2 instance"
  value       = aws_instance.devops_school_level3.public_dns
}
```

terraform tfvars

```
aws_region = "eu-north-1"
ami_id     = "ami-06fb1802e2b757f09"
subnet_id  = "subnet-05d4dca4820d2eb94"
main_sg_id = "sg-0fb87d0f08a213466"
ec2_type   = "t3.micro"
ec2_name   = "devops-school-level3.tabish-admin"
keypair    = "devops.school.level3.tabish-admin"
```

Appendix server commands

The following commands appear in the server history and were run after the SSH login.

```
sudo apt-get update && \
  sudo DEBIAN_FRONTEND=noninteractive apt-get upgrade -y
sudo DEBIAN_FRONTEND=noninteractive apt-get install -y nginx git
sudo systemctl enable --now nginx
systemctl is-active nginx
systemctl is-enabled nginx
git clone https://github.com/nenemustafa/Hellome.git \
  ~/hello-web && \
  ls -la ~/hello-web
sudo cp ~/hello-web/heyme /var/www/html/index.html && \
  sudo nginx -t && \
  sudo systemctl reload nginx
curl -i http://localhost/
exit
```

Local Terraform and SSH commands

```
cd ~/aws-devops-level3-iteration1
terraform fmt
terraform init
terraform validate
terraform plan -out=tfplan
terraform apply tfplan
MY_EC2=$(terraform output -raw ec2_public_dns_name)
ssh -o StrictHostKeyChecking=accept-new \
  -o IdentitiesOnly=yes -i ~/.ssh/id_rsa_level3 "ubuntu@$MY_EC2"
```

After the server work, the page was verified from WSL, followed by terraform plan, the removal plan and the final state check documented earlier in this report.